

Graph Theory Homework

In this assignment you will use the provided code template to implement solutions to each given problem. The solutions you submit must follow the guidelines provided in this document as well as any given in code.

Restrictions

- Template code is provided and must be used.
 - ***Note: The templates are refreshed each semester. Be sure to use the current version.***
- Your code must be compatible with **Python 3.10**.
- Do not add imports for any libraries beyond the Python standard library.
- Do not modify the function name or definition.

Testing

For each problem, one or more base test cases are provided. A properly implemented solution will pass any provided base test cases, but their trivial nature is not intended to ensure the overall correctness of your algorithm.

These are the same test cases that will be used to provide assurance that your submitted solutions run using the autograder. We do not release any other test cases.

In problem instances where more than one solution is optimal, *any* optimal solution will be considered correct, assuming that is the solution your algorithm produces.

The test cases can - and should! - be expanded while developing your solutions.

You can run your test cases by issuing the `python3 -m unittest` command from the directory containing your solution files.

Submission

Submit only - and all of - your solution files (i.e., `cs6515_[problem_name].py`) to the Gradescope assignment on or before the posted due date. Do not submit a zip file or any other files except those matching the naming convention defined.

You may submit your solutions as many times as desired and the most recent submission is what will be graded. Execution of your code may take up to 60 minutes. If not completed before the due date, your previous submission will be used.

Faster and correct solutions are worth more credit.

After your submission completes execution, you will see a score of - / 20 listed until grading is completed.

Lazy Bike Ride

Your least active TA, [redacted], has been told they need to exercise more. It's proposed that they go for a daily bike ride to visit all CS6515 students in their home city. The TA first decides they want to find a shortest path such that all students are visited. However, they (hopefully) quickly realize that finding such a path would be very difficult.

So instead they devise a new plan. They will rest after each visit to regain their energy until they are ready to ride again. Therefore, the total distance traveled is not an issue. Instead the TA looks to take the shortest routes between each student's home so they don't have to ride for long periods at a time.

You are provided a graph of the homes of the students in the city. Each vertex represents a home for the TA to visit and rest. Each edge represents either a bike route (E_b) that the TA can take between homes, or a non bike route (E_x) that the TA can take if they have to. The non bike route should be avoided **if possible** due to safety concerns.

Design an algorithm to produce a list of the best routes between student homes such that the TA visits all students.

Notes:

- A “route” is represented by a single Edge. So, a route between ‘a’ and ‘b’ would be (a, b).
- All homes are reachable.

Provide your code for implementing this algorithm in `cs6515_lazy_bike_ride.py`